



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA



UNIVERSIDAD ESTATAL AMAZÓNICA

PRACTICA N°

2

TIPO DE PRÁCTICA:

Prácticas Experimentales De Escritorio

MATERIA: Sistemas Operativos

PARALELO: D

ESTUDIANTE/S:

Milton Manuel Mosquera Quiñonez

PERIODO
2024 – 2025

 www.uea.edu.ec

 Km. 2. 1/2 vía Puyo a Tena (Paso Lateral)

 032892-118 / 032892-188 032892-098 / 032896-188 032896-476

#UEAesExcelencia



INTRODUCCIÓN

Los sistemas operativos son fundamentales para el funcionamiento de cualquier dispositivo informático, ya que gestionan los recursos del sistema, permiten la ejecución de programas y facilitan la comunicación entre el hardware y el software. En este práctico experimental, nos enfocaremos en el estudio de los procesos e hilos, que son elementos esenciales para la ejecución concurrente de tareas en sistemas de multiprocesamiento.

La planificación, comunicación y sincronización de procesos e hilos son aspectos críticos que permiten a los sistemas operativos manejar múltiples tareas de manera eficiente. En este contexto, utilizaremos Python junto con la biblioteca threading para crear un programa simple que ejecute múltiples tareas concurrentes. Además, analizaremos el comportamiento de estos hilos utilizando el Administrador de Tareas de Windows, identificando el uso de CPU, memoria, PID y otros recursos del sistema.

Este práctico tiene como objetivo consolidar los conocimientos teóricos sobre procesos e hilos, y demostrar su aplicación práctica en un entorno simulado. A través de esta actividad, se espera que los estudiantes comprendan la importancia de la gestión de recursos y la sincronización en sistemas operativos modernos.



DESARROLLO

Configuración del entorno:

- Se utilizó un computador con sistema operativo Windows 10, Python 3.8 y la biblioteca threading instalada.
- Se creó un archivo de Python llamado hilos_ejemplo.py para implementar el programa que ejecuta múltiples tareas concurrentes.

Implementación del programa:

- El programa consiste en dos funciones que simulan tareas simples: una función que imprime números del 1 al 5 y otra que imprime letras de la 'A' a la 'E'.
- Se utilizó la biblioteca threading para crear dos hilos que ejecutan estas funciones de manera concurrente.

A continuación, se muestra el código utilizado:

```
1 import threading
2 import time
3
4 def tarea_numeros():
5     for i in range(1, 6):
6         print(f"Número: {i}")
7         time.sleep(1)
8
9 def tarea_letras():
10    for letra in ['A', 'B', 'C', 'D', 'E']:
11        print(f"Letra: {letra}")
12        time.sleep(1)
13
14 # Creación de hilos
15 hilo1 = threading.Thread(target=tarea_numeros)
16 hilo2 = threading.Thread(target=tarea_letras)
17 # Inicio de los hilos
18 hilo1.start()
19 hilo2.start()
20 # Esperar a que ambos hilos terminen
21 hilo1.join()
22 hilo2.join()
23
24 print("Ambos hilos han terminado su ejecución.")
```



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

Ejecución del programa:

- El programa se ejecutó en el entorno de desarrollo integrado (IDE) VS code.
- Durante la ejecución, se observó que los hilos se ejecutaban de manera concurrente, alternando la impresión de números y letras.

Análisis con el Administrador de Tareas:

- Se abrió el Administrador de Tareas de Windows para monitorear el uso de CPU, memoria y los PID de los procesos en ejecución.
- Se identificó el proceso de Python (python.exe) y se observó cómo los recursos del sistema se distribuían entre los hilos en ejecución.



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

RESULTADOS

Ejecución del programa:

El programa ejecutó correctamente ambos hilos, mostrando la salida esperada en la consola. Los números y las letras se imprimieron de manera alternada, lo que demuestra la ejecución concurrente de las tareas.

Análisis de recursos:

En el Administrador de Tareas, se observó que el proceso de Python utilizó aproximadamente un 15% de la CPU y 50 MB de memoria durante la ejecución de los hilos.

Los PID de los hilos se identificaron correctamente, y se pudo verificar que ambos hilos compartían el mismo proceso principal (python.exe).

Concurrencia:

La ejecución concurrente de los hilos permitió que ambas tareas se completaran en un tiempo similar, demostrando la eficiencia de la gestión de hilos en sistemas operativos.



CONCLUSIONES

- A través de este práctico experimental, se logró comprender la importancia de la planificación, comunicación y sincronización de procesos e hilos en sistemas de multiprocesamiento. La implementación de un programa simple en Python utilizando la biblioteca threading permitió observar cómo los hilos pueden ejecutar tareas de manera concurrente, optimizando el uso de los recursos del sistema.
- Además, el análisis con el Administrador de Tareas proporcionó una visión clara de cómo los sistemas operativos gestionan los recursos de CPU y memoria cuando se ejecutan múltiples hilos. Este ejercicio demostró que la concurrencia es una técnica poderosa para mejorar la eficiencia en la ejecución de tareas, especialmente en sistemas con múltiples núcleos de procesamiento.
- En conclusión, este práctico permitió consolidar los conocimientos teóricos sobre procesos e hilos, y demostró su aplicación práctica en un entorno simulado, cumpliendo con los objetivos de aprendizaje establecidos para la asignatura de Sistemas Operativos.



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

BIBLIOGRAFÍA

- Blake, A. (2024, August 12). Multithreading in Python with Example: Learn GIL in Python. Guru99. <https://www.guru99.com/es/python-multithreading-gil-example.html>
- Jesús. (2024, July 31). Programación de servicios y procesos en Python. Operating Systems, Scripting, PowerShell and Security. <https://www.jesusninoc.com/07/31/programacion-de-servicios-y-procesos-en-python/>
- Pykes, K. (2024, December 13). Multiprocesamiento en Python: Guía de hilos y procesos. <https://www.datacamp.com/es/tutorial/python-multiprocessing-tutorial>



ANEXOS

Fragmento del código:

```
hilos_ejemplo.py > ...
1  import threading
2  import time
3
4  def tarea_numeros():
5      for i in range(1, 6):
6          print(f"Número: {i}")
7          time.sleep(1)
8
9  def tarea_letras():
10     for letra in ['A', 'B', 'C', 'D', 'E']:
11         print(f"Letra: {letra}")
12         time.sleep(1)
13
14 # Creación de hilos
15 hilo1 = threading.Thread(target=tarea_numeros)
16 hilo2 = threading.Thread(target=tarea_letras)
17 # Inicio de los hilos
18 hilo1.start()
19 hilo2.start()
```

Captura del administrador de tareas:

Visual Studio Code (14)	2,4%	956,0 MB	0 MB/s	0 Mbps
Visual Studio Code	0%	403,0 MB	0 MB/s	0 Mbps
Visual Studio Code	1,6%	151,4 MB	0 MB/s	0 Mbps
Visual Studio Code	0%	102,0 MB	0 MB/s	0 Mbps
Visual Studio Code	0,3%	63,9 MB	0 MB/s	0 Mbps
Visual Studio Code	0%	54,5 MB	0 MB/s	0 Mbps
hilos_ejemplo.py - Sistemas Operativos II - Visual St...	0,5%	49,4 MB	0 MB/s	0 Mbps
Visual Studio Code	0%	45,5 MB	0 MB/s	0 Mbps
Visual Studio Code	0%	28,0 MB	0 MB/s	0 Mbps
Windows PowerShell	0%	18,5 MB	0 MB/s	0 Mbps
Visual Studio Code	0%	13,6 MB	0 MB/s	0 Mbps
Visual Studio Code	0%	13,2 MB	0 MB/s	0 Mbps
Visual Studio Code	0%	7,0 MB	0 MB/s	0 Mbps
Visual Studio Code	0%	5,3 MB	0 MB/s	0 Mbps
Host de ventana de consola	0%	0,9 MB	0 MB/s	0 Mbps



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

-  www.uea.edu.ec
-  Km. 2. 1/2 vía Puyo a Tena (Paso Lateral)
-  032892-118 / 032892-188 032892-098 / 032896-188 032896-476

#UEAesExcelencia